What We Are Building, and What Has Happened So Far

BGSW AI Hackathon | AI-Driven Chaos Engineering | Status as of 10 October 2026

In one sentence: we are building a robot helper that deliberately breaks small parts of an online shop, watches what goes wrong, puts things back, and then explains the weak spots and how to fix them.

1. The Big Idea

Every app looks fine when nothing is going wrong. The real question is: **what happens when something breaks?** Does the whole site crash? Do customers lose their shopping? How long until it is working again?

Chaos engineering answers this by breaking things *on purpose*, in a safe practice area, before they break by accident in real life. It is like a fire drill: you set off the alarm on purpose so you learn where the exits are blocked, before there is a real fire.

The hackathon wants the breaking to be **smart**: an **AI** should decide *what* to break first, a **safety system** should stop things if they get too bad, and the AI should explain *what it learned*.

2. The Restaurant Picture

Software words are confusing, so think of the whole setup as a restaurant:

Real thing	In the restaurant	What it does
Online Boutique (the shop app)	The restaurant	A sample online shop made by Google. We picked it because it is free, public and made of many small parts.
The 11 services	The cooks	Each does one job: product list, cart, payment, shipping, emails, ads, prices in other currencies, and so on.
redis-cart	The fridge	A small database that remembers what is in every shopper's cart.
Kubernetes	The manager	Starts every cook, watches them, replaces anyone who stops working, and helps them talk to each other.
kind	A practice kitchen	A tiny Kubernetes that runs on your own laptop instead of in a big data centre.
Docker	The building	Runs the 'containers' (sealed boxes) that each program lives in.
WSL2	The land the building stands on	A small hidden Linux system inside Windows. Docker needs it, because containers are a Linux invention.

Our job: **unplug the fridge, make a cook go home, slow down a waiter**, and see whether the restaurant keeps serving customers.

3. What Happened, Step by Step

Step	What happened	In plain words
1. Tools	You tried *brew install*. It failed.	*brew* only works on Mac and Linux. On Windows we used *winget* instead. The main tools (kind, kubectl, helm) turned out to be installed already.
2. Shop code	Downloaded the Online Boutique code from GitHub.	We copied Google's shop onto your laptop.
3. Docker + WSL2	Docker Desktop installed, but would not start: "WSL not installed".	The building was ready, but there was no land under it.
4. The WSL saga	*wsl --install* kept sitting silently. We rebooted, tried again, then downloaded the official installer directly (351 MB).	It was not broken, just slow: your internet took about 15 minutes for that one file, and the command showed no progress bar. We checked the file was really from Microsoft before installing it.
5. Cluster	Created the practice kitchen (kind) and started all 11 services.	The restaurant opened. Everything showed 'Running'.
6. December bug	You spotted the card-expiry dropdown had two Januarys and no December.	A real typo in Google's code. We fixed it and swapped the fixed page into the running shop without rebuilding anything. Note it in your submission.
7. Map	Built the dependency map automatically from the shop's settings.	A list of which cook needs which other cook.
8. Baseline	Measured the shop while healthy: 1,940 requests, 0% errors, 95% of pages in under 26 milliseconds.	Took the restaurant's pulse on a normal day, so we can tell when something is wrong later.
9. Experiments	Broke three different services on purpose, with safety on.	See the next page.
10. The AI brain	Wrote the AI planner. Now installing a free AI model (Ollama).	See section 6.

4. The Experiments We Ran

We broke...	What happened	What it teaches
productcatalogservice (the product list)	100% of requests failed within 3 seconds. The safety system stopped the test by itself after about 6 seconds. Healthy again 18 seconds later.	Even the cart page dies when the product list is down. There is no backup plan or friendly message.
adservice (adverts)	No errors, but every page got about 100 ms slower. We pressed the kill switch by hand after 12 seconds.	Ads are optional, yet the site still waits for them on every page.
redis-cart (the cart fridge)	Pages froze for up to 5 seconds. Even browsing broke. A test cart with 6 items had **0 items** afterwards. Safety stopped it after about 6 seconds.	**Biggest finding.** All carts are erased whenever the cart database restarts, and the site shows no error, so nobody notices.

Why did the carts vanish? The cart database keeps everything in a temporary folder (*emptyDir*) that is deleted when it stops, like a fridge with no door seal: unplug it and everything inside is thrown away. Refreshing the page did not empty the cart; it only *showed* you that the data was already gone.

Why did browsing break when only the cart was down? Every page shows a little cart counter in the corner. To draw it, the page asks the cart service, so one small part going down dragged the whole site down with it. The settings files did not reveal this; the experiment did.

5. The Safety System

Breaking things is only useful if it is **safe**. Every experiment goes through a safety controller, like a lifeguard at the pool:

- **Checks the request first.** Only 11 named services in our own practice area may be touched. We tried 8 bad requests (another system, the load generator, a 10-minute test, even a sneaky command typed as a fault name). All 8 were refused before anything happened.
- **Checks the shop is healthy before starting.** It never breaks an already-sick shop.
- **Watches every 3 seconds.** If more than 10% of requests fail, or pages get too slow, two checks in a row, it stops the experiment automatically.
- **Kill switch.** A human can stop everything at any moment (*python -m chaos.run stop*).
- **Time limit.** No fault ever lasts more than 30 seconds.
- **Always cleans up**, even if something crashes, then measures how long the shop takes to recover.
- **Keeps evidence.** Every run is saved with times, numbers and results in *out/runs/*.

6. The AI Brain: Why Claude AND Ollama?

The hackathon requires that an **AI** chooses which experiments to run and explains why. So we need an AI model to think with. There are two ways to get one, and this is the part that caused confusion:

	Claude (online)	Ollama (on your laptop)
Like...	Phoning an expert consultant	Hiring a junior helper who lives in your office
Needs	An API key (a password from Anthropic) and a little credit	No key, but a big download (the program plus a 2.5 GB model)
Cost	A few cents per plan	Free
Quality	Very strong reasoning	Decent for its size, weaker than Claude
Laptop load	None, it runs on Anthropic's computers	Uses your graphics card (RTX 4050)

What we actually did: we wrote the AI planner so it can use *either one*, like a phone that takes two SIM cards. You did not have a Claude key, so we are **downloading Ollama** to use the free, local option. We are **not** using both at once. If you ever add a Claude key, the program automatically switches to Claude; until then it uses Ollama.

Why reinstall Ollama? Ollama was already on your laptop, but its program file was damaged (Windows called it 'corrupted and unreadable'), so it has to be installed again.

> **Important: whichever AI is used, it can only *suggest*. The safety system checks every suggestion and throws away anything unsafe. The AI never types commands into your computer.**

7. What Is Left To Do

Task	Status
Finish installing Ollama and download the AI model (qwen3:4b)	Downloading now
Let the AI pick and run experiments, with reasons	Code ready, waiting for the model
Two more types of failure: **network delay** (slow, not dead) and **traffic spike** (lots of fake shoppers at once). The rules need at least 3 types.	To do
AI-written findings report: weak spots, ranked, with fixes	To do
README, one-page architecture overview, demo video under 3 minutes, team form	To do
Optional: fix the cart database, re-run, and show carts survive	Nice to have

8. Mini Dictionary

Word	Meaning
Service	One small program that does one job in the shop.
Container	A sealed box holding a program and everything it needs to run.
Pod	Kubernetes' name for a running container.
Cluster	The group of computers (here, a pretend one on your laptop) that Kubernetes manages.
Namespace	A fenced-off area inside the cluster. Ours is called 'boutique'; the AI may only touch things inside it.
Baseline	Normal measurements taken while everything is healthy, for comparison.
p95 latency	95% of pages loaded faster than this. A way of saying 'almost always this fast'.
Kill switch	The emergency stop button.
API key	A secret password that lets a program use an online service such as Claude.
Port-forward	A tunnel that lets your browser reach the shop at http://localhost:8080.